Содержание

1 Векторы **1**
1.1 Вектор как структура данных, типы данных и приведение типов . . . 1
1.2 Извлечение элементов вектора . . . 2
1.3 Объединение векторов . . . 4
1.4 Последовательности . . . 4
1.5 Встроенные векторы . . . 5
1.6 Формат дата-время и его сочетание с последовательностями . . . 6
1.7 Векторы и конкатенация строк . . . 9

2 Матрицы **9**
2.1 Матрица как структура данных . . . 9
2.2 Извлечение элементов матрицы . . . 10
2.3 Операции со строками и столбцами матрицы . . . 11

3 Списки **12**
3.1 Список как структура данных . . . 12
3.2 Извлечение элементов списка . . . 13

4 Цикл `for` и разбиение строк **14**

Основы программирования в R: часть 1

1. Векторы

1.1. Вектор как структура данных, типы данных и приведение типов

Вектор — основная структура данных в R. Он представляет собой упорядоченный набор элементов одного типа. Если проводить аналогию со структурами данных в Python, вектор можно сравнивать с массивом (чаще одномерным, хотя создать сложный «вектор векторов» тоже можно).

Зачем изучать работу с векторами, если обычно в реальной жизни мы работаем с таблицами? Когда мы загружаем данные в табличном виде из файла в R, каждый столбец в таблице воспринимается R как отдельный вектор определённого типа (числового, текстового или логического). Да и при извлечении некоторых элементов из результатов статистических тестов так или иначе придётся иметь дело с векторами.

Создаётся вектор с помощью функции `c()`, от *combine*. Эта функция объединяет элементы в вектор. Создадим вектор `v1`, состоящий из целых чисел:

```
v1 <- c(3, 5, 8, 9)
v1

## [1] 3 5 8 9
```

Примечание: оператор `<-` используется для присваивания значений переменным (как `=` в Python и других языках). Оператор `=` тоже можно использовать, технически это не вызовет проблем, но все-таки это считается в R неправильным с точки зрения стиля кода:

```
v1 = c(3, 5, 8, 9)
v1

## [1] 3 5 8 9
```

Так как вектор умеет хранить элементы только одного типа, при наличии разных типов внутри вектора один самый сильный тип неизбежно вытеснит другие. Например, числовой тип (*numeric*, вещественные числа, то есть целые и дробные) сильнее целочисленного типа (*integer*):

```
v2 <- c(3, 6.6, 8, 9)
v2

## [1] 3.0 6.6 8.0 9.0
```

Из-за числа 6.6 все значения стали дробными (обратите внимание, в качестве десятичного разделителя в R используется точка). А текстовый тип (*character*), в свою очередь, сильнее числового и целочисленного:

```
v3 <- c("2,5", "3", 8, 9)
v3

## [1] "2,5" "3"   "8"   "9"
```

Проверим типы созданных векторов. Функция `class()` возвращает класс объекта в R, а функция `typeof()` — более точную спецификацию типа:

```
class(v1); typeof(v1)

## [1] "numeric"
## [1] "double"

class(v2); typeof(v2)

## [1] "numeric"
## [1] "double"

class(v3); typeof(v3)

## [1] "character"
## [1] "character"
```

Примечание: если перечислять какие-то команды через `;` в одну строку, они будут исполняться сразу друг за другом.

Теперь давайте выполним приведение типов — сконвертируем один тип в другой. Это понадобится нам позже для обработки реальных данных — если столбец имеет неверный тип, это может смущать (при чтении файла что-то пошло не так, и целочисленные коды ответов в опросе стали дробными) или мешать при построении графиков и применении подходящих методов анализа.

Сделаем вектор `v1` целочисленным (там и так целые числа, но R по умолчанию создал вектор более общего типа *numeric*) и сохраним результат конвертации в вектор `v4`. Воспользуемся функцией `as.integer()`:

```
v4 <- as.integer(v1)
v4

## [1] 3 5 8 9

class(v4); typeof(v4)

## [1] "integer"
## [1] "integer"
```

Все функции для приведения типов имеют название, состоящее из префикса `as` и названия типа. Проверим это и сделаем вектор `v1` текстовым:

```
v5 <- as.character(v1)
v5

## [1] "3" "5" "8" "9"
```

Теперь посмотрим, как работать с отдельными элементами вектора.

1.2. Извлечение элементов вектора

Так как вектор — упорядоченная структура, элементы вектора извлекаются по индексу (порядковому номеру). В отличие от Python, в R нумерация элементов начинается с 1, а не с 0.

Создадим более длинный вектор `long` и извлечём из него первый элемент:

```
long <- c(1, 9, 8, 4, 1, 9, 8, 5)
long[1]

## [1] 1
```

Если вызовем элемент с номером 0, получим «пустой» ответ:

```
long[0]

## numeric(0)
```

Последний элемент можно вызвать, определив длину вектора — число элементов в нём:

```
length(long)

## [1] 8

long[length(long)]

## [1] 5
```

Если нас интересует не один элемент, а сразу несколько, идущих подряд, выбрать их можно с помощью срезов:

```
long[2:4]

## [1] 9 8 4
```

Примечание: в отличие от Python, в R правый конец среза тоже включается в итоговый вектор выбранных элементов (здесь со второго элемента до четвёртого включительно).

Также, в отличие от Python, в R нельзя исключить какой-либо из концов среза, это приведёт к ошибке синтаксиса (`unexpected ']'`):

```
# long[2:]
```

Если нужны элементы со второго до последнего, придётся снова обращаться к длине вектора:

```
long[2:length(long)]

## [1] 9 8 4 1 9 8 5
```

А что будет, если мы запросим элементы со второго по девятый (всего элементов восемь)?

```
long[2:9]

## [1]  9  8  4  1  9  8  5 NA
```

R не выдаст ошибку, но на место несуществующего девятого элемента допишет `NA` — пустое значение (от *not applicable*). Как можно догадаться, если мы запросим элементы со второго по десятый, в конце вектора уже будет два `NA`:

```
long[2:10]

## [1]  9  8  4  1  9  8  5 NA NA
```

Если нам нужны элементы, стоящие не подряд, их индексы нужно объединять в вектор:

```
# 2nd, 5th, 8th
long[c(2, 5, 8)]

## [1] 9 1 5
```

Объединять обычные вектора и срезы при этом тоже можно:

```
long[c(2, 5:8)]

## [1] 9 1 9 8 5
```

```
long[c(2, 5, 1:6)]

## [1] 9 1 1 9 8 4 1 9
```

А можно ли в R, как в Python, использовать отрицательные индексы? Можно, но смысл такой операции будет иной, R не умеет считать элементы с конца. Попробуем!

```
long[-1]

## [1] 9 8 4 1 9 8 5
```

Что сделал R? Он выбрал все элементы, кроме первого! Так и работают отрицательные индексы, минус перед индексом означает, что нужно выбрать все элементы, кроме указанного. Проверим на втором элементе:

```
long[-2]

## [1] 1 8 4 1 9 8 5
```

А теперь проверим на срезе:

```
# long[-2:length(long)]
```

Не работает — сочетать отрицательные индексы можно только с нулём:

```
long[-2:0]

## [1] 8 4 1 9 8 5
```

Получили все элементы, кроме первых двух. Два отрицательных индекса в срезе — тоже нормальная ситуация:

```
long[-2:-4]

## [1] 1 1 9 8 5
```

Так, здесь мы выбрали все элементы, кроме второго, третьего и четвёртого.

1.3. Объединение векторов

Не исключено, что по аналогии с Python захочется склеивать векторы с помощью оператора +. Однако в R этот оператор используется для поэлементного сложения векторов:

```
c(1, 2, 3) + c(1, 0, 0)

## [1] 2 2 3
```

Если длины векторов разные, R выведет сообщение с предупреждением и сделает довольно неожиданную вещь — когда элементы более короткого вектора «закончатся», R начнет проходить этот вектор с самого начала. Рассмотрим пример:

```
c(1, 2, 3) + c(1, 2)

## Warning in c(1, 2, 3) + c(1, 2): длина большего объекта не является произведением длины меньшего объекта

## [1] 2 4 4
```

Почему в конце мы получили число 4? В первом векторе последний элемент складывать не с чем, во втором векторе всего два элемента. Поэтому R возвращается к началу второго вектора и складывает 3 с его первым элементом 1. Убедимся в этом на другом примере:

```
c(3, 7, 8, 10) + c(1, 2)

## [1]  4  9  9 12
```

А всё-таки, как объединить несколько векторов в один? Воспользоваться уже знакомой функцией `c()`:

```
c(c(1, 2, 3), c(1, 0))

## [1] 1 2 3 1 0
```

1.4. Последовательности

Если мы хотим получить векторы из целых чисел, идущих друг за другом, можно воспользоваться оператором `:` (аналог функции `range()` в Python с той разницей, что здесь правый конец среза тоже включается):

```
1:10

##  [1]  1  2  3  4  5  6  7  8  9 10
```

```
25:40

##  [1] 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
```

А если мы хотим получить наборы чисел с шагом, отличным от 1, понадобится функция `seq()` (от *sequence*). Первым аргументом этой функции будет начальное значение, вторым — конечное, а третьим — шаг:

```
seq(1, 20, 2)

##  [1]  1  3  5  7  9 11 13 15 17 19
```

```
seq(10, 15, 0.5)

##  [1] 10.0 10.5 11.0 11.5 12.0 12.5 13.0 13.5 14.0 14.5 15.0
```

Если шаг нам не известен (или нам не хочется его вычислять в уме), но мы знаем, сколько точек на одинаковом расстоянии друг от друга мы хотим получить в итоге, это число точек можно указать в аргументе `length.out`. Тогда R сам вычислит необходимый шаг и вернёт соответствующий вектор:

```
seq(1, 20, length.out = 10)

##  [1]  1.000000  3.111111  5.222222  7.333333  9.444444 11.555556 13.666667
##  [8] 15.777778 17.888889 20.000000
```

В данном случае, чтобы получить 10 равноудаленных друг от друга значений на отрезке от 1 до 20, R сам вычислил шаг 2.1111... и использовал его для построения последовательности. Если бы мы считали такой шаг самостоятельно, мы бы определили, что между 10 точками должно быть 9 промежутков, и поделили длину отрезка от 1 до 20, равную 19, на 9.

Для получения векторов из одинаковых значений нам пригодится функция `rep()` (от *repeat*):

```
rep("A", 3)

## [1] "A" "A" "A"
```

```
rep(NA, 10)

##  [1] NA NA NA NA NA NA NA NA NA NA
```

При этом повторять можно не только отдельные значения, но и векторы. Например, если для какой-нибудь столбиковой диаграммы нам понадобится набор двух чередующихся цветов, такой набор можно получить с помощью `rep()`:

```r
rep(c("red", "blue"), 4)

## [1] "red"  "blue" "red"  "blue" "red"  "blue" "red"  "blue"
```

А если цвета не чередуются, а просто повторяются фиксированное число раз друг за другом, потребуется аргумент `each`:

```r
rep(c("red", "blue"), each = 4)

## [1] "red"  "red"  "red"  "red"  "blue" "blue" "blue" "blue"
```

1.5. Встроенные векторы

Также в R есть уже готовые векторы и последовательности, которые могут быть полезны. Например, вектор со строчными буквами английского алфавита:

```r
letters

##  [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"
## [20] "t" "u" "v" "w" "x" "y" "z"
```

Или с заглавными буквами английского алфавита:

```r
LETTERS

##  [1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S"
## [20] "T" "U" "V" "W" "X" "Y" "Z"
```

Вектор с названиями месяцев на английском языке:

```r
month.name

##  [1] "January"   "February"  "March"     "April"     "May"       "June"
##  [7] "July"      "August"    "September" "October"   "November"  "December"
```

Вектор с сокращёнными названиями месяцев на английском языке:

```r
month.abb

##  [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"
```

Также можно получить вектор с названиями цветов, которые используются в R (правда, `colors()` — это уже не готовый вектор, это функция, которая такой вектор возвращает):

```r
# first 10 to be short
colors()[1:10]

##  [1] "white"         "aliceblue"     "antiquewhite"  "antiquewhite1"
##  [5] "antiquewhite2" "antiquewhite3" "antiquewhite4" "aquamarine"
##  [9] "aquamarine1"   "aquamarine2"
```

Какие цвета соответствуют этим названиям, можно посмотреть здесь.

1.6. Формат дата-время и его сочетание с последовательностями

Раз уж заговорили о последовательностях, имеет смысл поговорить о создании полезных для реальной обработки данных последовательных, связанных со специальным форматом *дата-время*. Технически, даты и метки времени можно хранить в виде обычного текста, но если мы захотим данные с такими метками сортировать или визуализировать, неизбежно возникнут проблемы — строки будут сортироваться по алфавиту, а не хронологически.

Для создания набора дат с нуля нам понадобится функция `ISOdate()`. Она забирает формат даты и времени, используемый в системе, на компьютере, и возвращает последовательность меток вида *дата-время*. Создадим вектор для дат, представляющих собой первые числа каждого месяца 2025 года:

```r
# year 2025
# months from 1 to 12
# day 1
# time 12:00:00 GMT by default

dates <- ISOdate(2025, 1:12, 1)
dates

##  [1] "2025-01-01 12:00:00 GMT" "2025-02-01 12:00:00 GMT"
##  [3] "2025-03-01 12:00:00 GMT" "2025-04-01 12:00:00 GMT"
##  [5] "2025-05-01 12:00:00 GMT" "2025-06-01 12:00:00 GMT"
##  [7] "2025-07-01 12:00:00 GMT" "2025-08-01 12:00:00 GMT"
##  [9] "2025-09-01 12:00:00 GMT" "2025-10-01 12:00:00 GMT"
## [11] "2025-11-01 12:00:00 GMT" "2025-12-01 12:00:00 GMT"
```

Проверим тип вектора:

```r
class(dates)

## [1] "POSIXct" "POSIXt"
```

POSIXct и *POSIXt* — специальные форматы для даты и времени. Метки с датой и временем, которые мы создали, с одной стороны, хранят дату в виде POSIX-времени (число секунд с 1 января 1970 года, универсальный формат хранения временных данных, используется на сайтах и в социальных сетях), а с другой стороны, позволяют увидеть её в более привычном текстовом виде.

Аналогичным образом создадим вектор со вторыми числами каждого месяца:

```r
ISOdate(2025, 1:12, 2)

##  [1] "2025-01-02 12:00:00 GMT" "2025-02-02 12:00:00 GMT"
##  [3] "2025-03-02 12:00:00 GMT" "2025-04-02 12:00:00 GMT"
##  [5] "2025-05-02 12:00:00 GMT" "2025-06-02 12:00:00 GMT"
##  [7] "2025-07-02 12:00:00 GMT" "2025-08-02 12:00:00 GMT"
##  [9] "2025-09-02 12:00:00 GMT" "2025-10-02 12:00:00 GMT"
## [11] "2025-11-02 12:00:00 GMT" "2025-12-02 12:00:00 GMT"
```

А вот с последними числами будет проблема, очевидно, что не в каждом месяце 31 день:

```r
ISOdate(2025, 1:12, 31)

##  [1] "2025-01-31 12:00:00 GMT" NA
##  [3] "2025-03-31 12:00:00 GMT" NA
##  [5] "2025-05-31 12:00:00 GMT" NA
##  [7] "2025-07-31 12:00:00 GMT" "2025-08-31 12:00:00 GMT"
##  [9] NA                        "2025-10-31 12:00:00 GMT"
## [11] NA                        "2025-12-31 12:00:00 GMT"
```

R на месте месяцев с другим числом дней добавил пропуски `NA`. Для корректного набора придется вместо 31 честно сформировать вектор с последними числами месяца:

```r
ISOdate(2025, 1:12, c(31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31))

##  [1] "2025-01-31 12:00:00 GMT" "2025-02-28 12:00:00 GMT"
##  [3] "2025-03-31 12:00:00 GMT" "2025-04-30 12:00:00 GMT"
##  [5] "2025-05-31 12:00:00 GMT" "2025-06-30 12:00:00 GMT"
##  [7] "2025-07-31 12:00:00 GMT" "2025-08-31 12:00:00 GMT"
##  [9] "2025-09-30 12:00:00 GMT" "2025-10-31 12:00:00 GMT"
## [11] "2025-11-30 12:00:00 GMT" "2025-12-31 12:00:00 GMT"
```

При этом, если мы хотим извлечь из метки с датой и временем, какую-то конкретную информацию в виде обычного текста, мы тоже сможем это сделать. Понадобится функция `format()`, она извлечет нужный элемент и подставит его в составленный нами текстовый шаблон.

Для начала извлечем дату в сокращённом виде.

```r
# before
dates

##  [1] "2025-01-01 12:00:00 GMT" "2025-02-01 12:00:00 GMT"
##  [3] "2025-03-01 12:00:00 GMT" "2025-04-01 12:00:00 GMT"
##  [5] "2025-05-01 12:00:00 GMT" "2025-06-01 12:00:00 GMT"
##  [7] "2025-07-01 12:00:00 GMT" "2025-08-01 12:00:00 GMT"
##  [9] "2025-09-01 12:00:00 GMT" "2025-10-01 12:00:00 GMT"
## [11] "2025-11-01 12:00:00 GMT" "2025-12-01 12:00:00 GMT"
```

```r
# after
format(dates, "%D")

##  [1] "01/01/25" "02/01/25" "03/01/25" "04/01/25" "05/01/25" "06/01/25"
##  [7] "07/01/25" "08/01/25" "09/01/25" "10/01/25" "11/01/25" "12/01/25"
```

Символ `%` сообщает R, что на это место нужно подставить некоторое значение (то, что мы указали в кавычках, и есть текстовый шаблон), а буква `D` соответствует дате. Текстовый шаблон мог быть и другим:

```r
format(dates, "Date is %D")

##  [1] "Date is 01/01/25" "Date is 02/01/25" "Date is 03/01/25" "Date is 04/01/25"
##  [5] "Date is 05/01/25" "Date is 06/01/25" "Date is 07/01/25" "Date is 08/01/25"
##  [9] "Date is 09/01/25" "Date is 10/01/25" "Date is 11/01/25" "Date is 12/01/25"
```

Помимо `D` есть много других полезных сокращений:

- `d`: число (номер дня);
- `B`: название месяца (на языке, используемом системой);
- `b`: сокращенное название месяца;
- `Y`: год в четырехзначном формате;
- `y`: год в двузначном формате (последние две цифры);
- `A`: название дня недели (на языке, используемом системой);
- `a`: сокращенное название дня недели;
- `H`: часы;
- `M`: минуты;
- `S`: секунды.

Посмотрим на примеры разных строк с разными элементами даты и времени (логика простая — на место `%` и буквы подставляется подходящий элемент, а сама строка может включать пробелы, скобки, двоеточия и прочие символы).

```r
format(dates, "%d %B %Y")

##  [1] "01 января 2025"   "01 февраля 2025"  "01 марта 2025"    "01 апреля 2025"
##  [5] "01 мая 2025"      "01 июня 2025"     "01 июля 2025"     "01 августа 2025"
##  [9] "01 сентября 2025" "01 октября 2025"  "01 ноября 2025"   "01 декабря 2025"
```

```r
format(dates, "%d %B %y")

##  [1] "01 января 25"   "01 февраля 25"  "01 марта 25"    "01 апреля 25"
##  [5] "01 мая 25"      "01 июня 25"     "01 июля 25"     "01 августа 25"
##  [9] "01 сентября 25" "01 октября 25"  "01 ноября 25"   "01 декабря 25"
```

```r
format(dates, "%d %b %y")

##  [1] "01 янв 25" "01 фев 25" "01 мар 25" "01 апр 25" "01 май 25" "01 июн 25"
##  [7] "01 июл 25" "01 авг 25" "01 сен 25" "01 окт 25" "01 ноя 25" "01 дек 25"
```

```r
format(dates, "%d %B %Y (%A)")

##  [1] "01 января 2025 (среда)"         "01 февраля 2025 (суббота)"
##  [3] "01 марта 2025 (суббота)"        "01 апреля 2025 (вторник)"
##  [5] "01 мая 2025 (четверг)"          "01 июня 2025 (воскресенье)"
##  [7] "01 июля 2025 (вторник)"         "01 августа 2025 (пятница)"
##  [9] "01 сентября 2025 (понедельник)" "01 октября 2025 (среда)"
## [11] "01 ноября 2025 (суббота)"       "01 декабря 2025 (понедельник)"
```

```r
format(dates, "%d %B %Y (%a)")

##  [1] "01 января 2025 (ср)"   "01 февраля 2025 (сб)"  "01 марта 2025 (сб)"
##  [4] "01 апреля 2025 (вт)"   "01 мая 2025 (чт)"      "01 июня 2025 (вс)"
##  [7] "01 июля 2025 (вт)"     "01 августа 2025 (пт)"  "01 сентября 2025 (пн)"
## [10] "01 октября 2025 (ср)"  "01 ноября 2025 (сб)"   "01 декабря 2025 (пн)"
```

```r
format(dates, "%d %B %Y %H:%M:%S")

##  [1] "01 января 2025 12:00:00"   "01 февраля 2025 12:00:00"
##  [3] "01 марта 2025 12:00:00"    "01 апреля 2025 12:00:00"
##  [5] "01 мая 2025 12:00:00"      "01 июня 2025 12:00:00"
##  [7] "01 июля 2025 12:00:00"     "01 августа 2025 12:00:00"
##  [9] "01 сентября 2025 12:00:00" "01 октября 2025 12:00:00"
## [11] "01 ноября 2025 12:00:00"   "01 декабря 2025 12:00:00"
```

1.7. Векторы и конкатенация строк

Конкатенация строк — склеивание строк, то есть приписывание одной строки в конец другой. Для склеивания строк в R используется функция `paste()`:

```r
paste("A", "B", "C")

## [1] "A B C"
```

По умолчанию в качестве разделителя используется пробел, но его можно поменять, добавив аргумент `sep` (от *separator*):

```r
paste("A", "B", "C", sep = "-")

## [1] "A-B-C"
```

Что замечательно, эта функция `paste()` умеет работать и с векторами тоже. То есть, используя эту функцию, к каждому элементу вектора можно доклеить какой-то текст, минуя циклы и подобные конструкции (операции в R векторизованы — применяются ко всем элементам вектора, здесь это и проявляется). Для примера доклеим к каждому числу в последовательности от 231 до 233 слово «группа»:

```r
paste("группа", 231:233)

## [1] "группа 231" "группа 232" "группа 233"
```

Или так:

```
paste("группа", 231:233, sep = "_")

## [1] "группа_231" "группа_232" "группа_233"
```

Тем, кто знаком с Python, история про склеивание строк неизбежно наводит на мысль о методе `.join()`. В R такой функции нет, объединение вектора строк в одну большую строку выполняет та же функция `paste()`, но с аргументом `collapse`, в котором необходимо указать желаемый разделитель:

```
parts <- c("12", "02", "2025")
paste(parts, collapse = "/")

## [1] "12/02/2025"
```

2. Матрицы

2.1. Матрица как структура данных

Матрица — двумерный массив в R, то есть таблица, состоящая из элементов одного типа. С матрицами можно столкнуться, изучая сетевой анализ (матрица смежности или матрица инцидентности для описания связей между людьми или странами) или создавая небольшие таблицы для хранения данных с нуля.

Способов создания матриц несколько, мы начнём с самого простого — разобьём набор чисел от 10 до 21 на 4 строки:

```
M <- matrix(10:21, nrow = 4)
M

##      [,1] [,2] [,3]
## [1,]   10   14   18
## [2,]   11   15   19
## [3,]   12   16   20
## [4,]   13   17   21
```

По умолчанию элементы записываются по столбцам матрицы (сначала заполняется первый столбец, потом второй, и так далее). Если хочется заполнять матрицу по строкам, потребуется аргумент `byrow`:

```
matrix(10:21, nrow = 4, byrow = TRUE)

##      [,1] [,2] [,3]
## [1,]   10   11   12
## [2,]   13   14   15
## [3,]   16   17   18
## [4,]   19   20   21
```

Вернемся к матрице `M` и столбцам названия её строкам:

```
rownames(M) <- month.abb[1:4]
M

##     [,1] [,2] [,3]
## Jan   10   14   18
## Feb   11   15   19
## Mar   12   16   20
## Apr   13   17   21
```

И заодно — названия столбцам:

```
colnames(M) <- c("Nif-Nif", "Naf-Naf", "Nuf-Nuf")
M

##     Nif-Nif Naf-Naf Nuf-Nuf
## Jan      10      14      18
```

```
## Feb      11      15      19
## Mar      12      16      20
## Apr      13      17      21
```

Будем считать, что в матрице выше сохранено число желудей, которые собрали поросята Ниф-Ниф, Наф-Наф и Нуф-Нуф в январе, феврале, марте и апреле. Конечно, в данном случае пример игрушечный, но функции `rownames()` и `colnames()` пригодятся нам позже при работе с реальными данными, потому что названия строк и столбцов в датафреймах (таблицах) фиксируются точно так же.

2.2. Извлечение элементов матрицы

Работа с элементами матрицы в R напоминает работу с элементами матрицы в математике — на первом месте всегда указывается номер строки, где находится элемент, на втором — номер столбца. Выберем элемент на пересечении первой строки и третьего столбца:

```
M[1, 3]

## [1] 18
```

А теперь — на пересечении третьей строки и первого столбца:

```
M[3, 1]

## [1] 12
```

Если мы хотим выбрать сразу несколько строк или столбцов, то есть «вырезать» маленькую матрицу из большой, индексы строк или столбцов нужно указывать в виде вектора (последовательности):

```
M[c(2, 4), 3]

## Feb Apr
##  19  21
```

```
M[1, 2:3]

## Naf-Naf Nuf-Nuf
##      14      18
```

2.3. Операции со строками и столбцами матрицы

Когда мы говорим о матрицах, почти всегда речь идёт о числовых матрицах. А для числовых матриц и таблиц в R есть удобные функции для подсчёта сумм и средних по строкам или столбцам. Посчитаем сумму по каждому столбцу:

```
colSums(M)

## Nif-Nif Naf-Naf Nuf-Nuf
##      46      62      78
```

Или по каждой строке:

```
rowSums(M)

## Jan Feb Mar Apr
##  42  45  48  51
```

Аналогичная история со средними значениями:

```
colMeans(M)

## Nif-Nif Naf-Naf Nuf-Nuf
##    11.5    15.5    19.5
```

```
rowMeans(M)

## Jan Feb Mar Apr
##  14  15  16  17
```

К сожалению, готовых функций для вычисления других характеристик (медиана, дисперсия, стандартное отклонение) по каждому столбцу или строке в R нет. Но, к счастью, есть более универсальная функция `apply()`, которая позволяет применять какую-то функцию к каждой строке или столбцу.

Так, вместо `rowSums(M)` мы могли бы записать следующее:

```
apply(M, 1, sum)

## Jan Feb Mar Apr
##  42  45  48  51
```

Это означает: примени функцию `sum()`, которая в R уже есть, к каждой строке в матрице `M` (1 — строки, 2 — столбцы).

Проделаем то же самое для столбцов:

```
apply(M, 2, sum)

## Nif-Nif Naf-Naf Nuf-Nuf
##      46      62      78
```

Вместо `sum` можно подставить любую другую функцию. Например, функцию для нахождения минимума:

```
apply(M, 2, min)

## Nif-Nif Naf-Naf Nuf-Nuf
##      10      14      18
```

Или дисперсии (здесь одинаковая у всех, данные такие):

```
apply(M, 2, var)

##  Nif-Nif  Naf-Naf  Nuf-Nuf
## 1.666667 1.666667 1.666667
```

Или стандартного отклонения:

```
apply(M, 2, sd)

##  Nif-Nif  Naf-Naf  Nuf-Nuf
## 1.290994 1.290994 1.290994
```

А можно вообще написать свою функцию (будем более подробно обсуждать позже), которая будет переводить значение в ячейке в проценты от столбца или строки.

Проценты от суммы по строке:

```
apply(M, 1, function(x){x / sum(x) * 100})

##              Jan      Feb      Mar      Apr
## Nif-Nif 23.80952 24.44444 25.00000 25.49020
## Naf-Naf 33.33333 33.33333 33.33333 33.33333
## Nuf-Nuf 42.85714 42.22222 41.66667 41.17647
```

Проценты от суммы по столбцу:

```
apply(M, 2, function(x){x / sum(x) * 100})

##      Nif-Nif  Naf-Naf  Nuf-Nuf
## Jan 21.73913 22.58065 23.07692
## Feb 23.91304 24.19355 24.35897
## Mar 26.08696 25.80645 25.64103
## Apr 28.26087 27.41935 26.92308
```

3. Списки

3.1. Список как структура данных

Список (*list*) в R — структура довольно специфическая, она одновременно напоминает как списки, так и словари в Python. Ключевое сходство со списками в Python заключается в том, что и те, и другие умеют хранить данные разных типов. А сходство со словарями заключается в том, что в самом простом случае список — это просто пара соответствий вида *ключ-значение*.

Вот с примера таких соответствий мы и начнём. Создадим список `pairs`, который содержит уникальные ответы респондентов и соответствующие им обозначения:

```
pairs <- list("Да" = "yes", "Нет" = "no", "Нет ответа" = NA)
pairs

## $Да
## [1] "yes"
##
## $Нет
## [1] "no"
##
## $`Нет ответа`
## [1] NA
```

Такого вида соответствия часто встречаются в реальной работе с данными. Например, опции в выпадающем меню, которые мы показываем пользователю в приложении, содержат понятные и аккуратно оформленные ответы вроде «Да» и «Нет», а в таблице с данными, откуда мы извлекаем данные по этим ответам, они записаны попроще и на латинице. Чтобы соединить выбор пользователя в меню с имеющимися данными, нам неизбежно потребуется сохранить доступные опции в виде пар связанных значений.

Также списки можно использовать для хранения информации в структурированном виде (если таблица не подходит в силу специфического вида данных):

```
L <- list(data = "students",
          names = c("Elaine", "Cassandra", "Harry"),
          marks = c(6, 8, 9))
L

## $data
## [1] "students"
##
## $names
## [1] "Elaine"    "Cassandra" "Harry"
##
## $marks
## [1] 6 8 9
```

3.2. Извлечение элементов списка

Как можно заметить, название каждого «крупного» элемента в списке записано после `$`. Действительно, этот оператор используется для извлечения элементов по названию — в датафреймах с реальными данными будет то же самое:

```
pairs$Да

## [1] "yes"
```

Если название содержит пробел или нетипичные для переменных символы, его нужно вводить в специальных кавычках:

```
pairs$`Нет ответа`

## [1] NA
```

Выбирать элементы по номеру тоже можно, как в векторах:

```
pairs[1]

## $Да
## [1] "yes"
```

Если мы указываем номер элемента в одинарных квадратных скобках, нам возвращается объект типа *список*, то есть маленький список из одного элемента. Однако если мы укажем номер элемента в двойных квадратных скобках, этот элемент будет «полностью» извлечен, то есть он будет иметь тот тип, который у него был изначально при добавлении в список.

```
# not a list, just a word
pairs[[1]]

## [1] "yes"
```

Сравним:

```
class(pairs[1])

## [1] "list"

class(pairs[[1]])

## [1] "character"
```

Если список обладает сложной вложенной структурой, выбор элементов может быть многоступенчатым: сначала выбираем один элемент, потом внутри него другой и так далее.

Выберем первый элемент из `names` в списке `L`:

```
L$names[1]

## [1] "Elaine"
```

А теперь проделаем то же самое, только с помощью числовых индексов:

```
L[[2]][1]

## [1] "Elaine"
```

4. Цикл `for` и разбиение строк

Обычно цикл `for` используется для повторения операций фиксированное число раз. В R цикл `for` выполняет ту же задачу, однако не всегда использование этого цикла действительно необходимо. Основной структурой данных в R является вектор, а это значит, что многие операции являются векторизованными: применяя операцию к вектору, мы автоматически применяем её сразу ко всем его элементам.

Например, для домножения всех элементов вектора на число цикл нам не понадобится:

```
vec <- c(-0.6, 0.1, 0.5, 0.8, 2, 3)
vec ** 2

## [1] 0.36 0.01 0.25 0.64 4.00 9.00
```

То же самое мы видели при работе с несколькими векторами одинаковой длины.

Однако, если мы работаем не с готовыми векторами, цикл `for` всё же понадобится. Например, для перебора файлов в какой-нибудь папке. Представим себе такую ситуацию: у нас есть набор CSV-файлов с одинаковой структурой (один и тот же опрос в разное время, одни и те же индексы по разным странам, результаты выборов в разных регионах), и мы хотим из каждого файла извлечь столбец с одним и тем же названием и построить для него какой-нибудь график. Тут-то и пригодится цикл `for`.

Давайте пока не будем брать файлы с данными, а просто создадим вектор с реалистичными названиями файлов:

```
files <- c("France.csv",
           "Spain.csv",
           "Slovenia.csv",
           "Brazil.csv")
```

Переберем названия файлов в цикле и выведем их названия на экран (как обычно в циклах, вместо `f` мы могли вписать любое название):

```
for (f in files){
  print(f)
}

## [1] "France.csv"
## [1] "Spain.csv"
## [1] "Slovenia.csv"
## [1] "Brazil.csv"
```

Теперь рассмотрим задачу поинтереснее. Предположим, дальше для данных из каждого файла мы будем строить графики. И мы хотим, чтобы заголовки графиков соответствовали названиям стран. Чтобы это автоматизировать, нам потребуется код, который будет извлекать название страны из названия CSV-файла. Начнем с одного названия, сохраним его как `f`:

```
f <- files[1]
```

Сначала название файла нужно разбить по точке. Для разбиения строки на части в R есть функция `strsplit()`:

```
strsplit(f, "\\.")

## [[1]]
## [1] "France" "csv"
```

Почему в функции выше вместо разделителя `.` у нас указано `\\.`? Многие функции для работы со строками в R автоматически поддерживают работу с регулярными выражениями для поиска и изменения текста по заданному шаблону, а в регулярных обозначениях точка является специальным символом. В регулярных выражениях точка означает любой символ. То есть, выражение `т.к` будет находить слова `ток`, `тик`, `так`, `тук` или даже `т9к`. Поэтому, чтобы R понимал точку как точку, её пришлось экранировать — выделить с помощью `\\`.

Функция `strsplit()` возвращает результат в виде списка.

Заберём из этого списка единственный вектор с индексом 1:

```
strsplit(f, "\\.")[[1]]

## [1] "France" "csv"
```

Теперь из этого вектора извлечём первый элемент — название страны и сделаем все буквы заглавными:

```
name <- toupper(strsplit(f, "\\.")[[1]][1])
name

## [1] "FRANCE"
```

Осталось применять эту операцию в цикле и посмотреть на результаты:

```
for (f in files){
  name <- toupper(strsplit(f, "\\.")[[1]][1])
  print(name)
}

## [1] "FRANCE"
## [1] "SPAIN"
## [1] "SLOVENIA"
## [1] "BRAZIL"
```

Отлично! Запомним эти манипуляции со строками, они нам понадобятся в практической части!